

Pairing Socks

Day 1 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“socks stacked in a pile”*
- ▶ *“least number of moves she needs to pair the socks”*

Programming interpretation:

- ▶ Model the original pile and auxiliary pile as stacks.
- ▶ A move transfers one top sock; a pair operation removes matching top socks.
- ▶ If the deterministic forward simulation empties both stacks, the move count is the answer.

Textbook connection: CP4 2.2 j. Stack

Core technique: Stack simulation

- ▶ The input order is a stack of socks.
- ▶ A second stack represents the auxiliary pile.
- ▶ A pair disappears when the top sock of each stack has the same color.

- ▶ Only the top of each pile matters, so stack operations are natural.
- ▶ Moving a sock back from auxiliary to original does not create a useful new order.
- ▶ The process is deterministic once we always move from original to auxiliary.

- ▶ Read the original stack and initialize an empty auxiliary stack.
- ▶ Pop one sock from the original stack and push it onto the auxiliary stack.
- ▶ After each move, repeatedly remove matching top socks from both stacks.
- ▶ Count every move or successful pair-removal action as required by the problem.
- ▶ If both stacks are empty, print the count; otherwise print impossible.

- ▶ The outer loop keeps feeding the auxiliary stack.
- ▶ The inner loop handles any chain reaction of exposed matching socks.
- ▶ A nonempty auxiliary stack at the end means some socks could not be paired legally.

- ▶ The algorithm simulates the only useful direction of movement.
- ▶ Whenever a pair is exposed, removing it immediately cannot block a future match because it only reveals deeper socks.
- ▶ If the process ends with leftover socks, no later legal action can pair them because the original stack is empty.

Complexity: $O(N)$ time and $O(N)$ memory.

- ▶ Python lists are sufficient stacks with append and pop.
- ▶ Use truthiness of lists to test whether a stack is empty.
- ▶ Be careful that input lists are read in the orientation expected by the simulation.

Source lines: pairingsocks.py:43-54

```
import sys

lines = sys.stdin.read().splitlines()
stack1 = lines[1].split()

sockPairs = int(lines[0])
moveCounter = 0
stack2 = []
```

What this chunk does

- ▶ The first line gives the number of pairs; the second line gives the stack order.
- ▶ stack1 is the original pile and stack2 starts empty.
- ▶ moveCounter tracks both transfers and pair removals.

Source lines: pairingsocks.py:56-63

```
while stack1:
    stack2.append(stack1.pop())
    moveCounter += 1

    while stack1 and stack2 and stack2[-1] == stack1[-1]:
        stack2.pop()
        stack1.pop()
        moveCounter += 1
```

What this chunk does

- ▶ Each loop moves one sock from the original pile to the auxiliary pile.
- ▶ Whenever the two top socks match, both are removed immediately.
- ▶ The inner loop handles chain reactions after exposing deeper socks.

Source lines: pairingsocks.py:65-68

```
if stack1 or stack2:  
    sys.stdout.write("impossible")  
else:  
    sys.stdout.write(str(moveCounter))
```

What this chunk does

- ▶ Any remaining sock means a complete legal pairing was not found.
- ▶ If both stacks are empty, every sock was paired and the counted moves are printed.

Solution file: `practice_contests/day_1/pairingsocks.py`

Problem: <https://open.kattis.com/problems/pairingsocks>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.